

Optimizing C Code Using LLVM: Case Studies and Performance Enhancements

Abhay Pratap Singh [CO23306]

May 1st 2025

□ Introduction

With the ever-increasing demand for performance in software applications, especially in systems that rely heavily on numerical computations or real-time processing, compiler-level optimizations have become indispensable. LLVM (Low-Level Virtual Machine) offers a powerful and flexible infrastructure for optimizing programs at an intermediate representation (IR) level. Unlike high-level source code optimization, LLVM IR-level optimization operates closer to machine-level instructions, offering greater control and potential for performance enhancement.

This project aims to explore how LLVM can be employed to optimize C code by compiling it into LLVM IR and applying various transformation and analysis passes. We demonstrate this process through a series of case studies involving common algorithms: matrix multiplication, bubble sort, Fibonacci sequence, and matrix addition.

□ Assumptions and Justifications

To ensure meaningful and measurable outcomes, we make the following assumptions:

- The original C code is written in a straightforward and naïve fashion, similar to code written by beginner to intermediate programmers. This helps highlight the effect of LLVM optimizations more clearly.
- The optimizations applied are standard LLVM passes, which are widely used and well-documented.
- Performance is evaluated using the Linux `time` utility, capturing `real`, `user`, and `sys` time for consistency and simplicity.
- Experiments are run in a controlled environment: same hardware, no background processes, warm CPU caches.
- Matrix sizes and loop iterations are chosen to be large enough to show significant timing differences post optimization.

These assumptions help isolate and showcase the effectiveness of LLVM passes without external variables influencing the outcome.

□ Architecture

→ System Overview

The optimization framework uses LLVM's modular compiler tools (`clang`, `opt`, and `llc`) combined with traditional `gcc` for linking and execution. The workflow involves converting C source code to LLVM IR, applying selected optimization passes, and compiling back to executable code for performance comparison.

→ Optimization Pipeline

1. Source Code Preparation

Write unoptimized C programs simulating real-world logic.

```
clang -S -emit-llvm code.c -o code.ll
```

2. IR Generation

Use Clang to generate LLVM IR from the C source:

```
clang -S -emit-llvm code.c -o code.ll
```

3. Apply Optimizations

Use `opt` with the new pass manager to apply transformations:

```
opt -passes="mem2reg,loop-unroll,instcombine,gvn" code.ll -S -o code_opt.ll
```

4. IR to Assembly

Use `opt` with the new pass manager to apply transformations:

```
llc code_opt.ll -o code_opt.s
```

5. Linking and Exevution

Compile to binary and test:

```
gcc code_opt.s -o code_opt  
time ./code_opt
```

→ LVM Passes Used

- *mem2reg*: Promotes stack memory variables to SSA form.
- *loop-unroll*: Reduces loop overhead by unrolling loop iterations.
- *instcombine*: Simplifies and merges redundant instructions.
- *gvn*: Eliminates common subexpressions.
- *inline*: Inlines function calls to reduce call overhead.

□ Methodology

The methodology centers around controlled experiments where each C program is compiled, optimized, and executed under identical conditions. For each example:

1. Unoptimized Execution

- Compile C code directly and time it using `time ./binary`.

2. Optimized Execution

- Apply selected LLVM passes and repeat the timing test.

3. Result Recording

- Capture `real`, `user`, and `sys` time for both versions.

4. Analysis

- Compare timing results and identify which optimizations contributed most.

All tests were run on an x86_64 Linux machine with an Intel i5 processor and 8GB RAM.

□ Experimentation

We explore three algorithms to evaluate the impact of LLVM optimizations.

1. Matrix Multiplication

Multiplies two 100x100 matrices.

C Code Highlights:

- Nested loops (i, j, k) used for computing dot products.

Applied Optimizations:

- *mem2reg*: Promotes scalar values.
- *loop-unroll*: Reduces loop branching.
- *gvn*, *instcombine*: Simplifies expressions.

Step 1: Write the C Code for Matrix Multiplication

The first step is to write a simple matrix multiplication function in C, which multiplies two square matrices and stores the result in a third matrix.

The code is given in file attached

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#define SIZE 100
```

```
void multiply_matrices(float A[SIZE][SIZE], float B[SIZE][SIZE], float C[SIZE][SIZE]) {
```

```
    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            C[i][j] = 0;
            for (int k = 0; k < SIZE; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

```
}
```

```
int main() {
```

```
    float A[SIZE][SIZE], B[SIZE][SIZE], C[SIZE][SIZE];
```

```
    srand(time(NULL));
```

```
    // Initialize matrices with random float values
```

```
    for (int i = 0; i < SIZE; i++) {
```

```

for (int j = 0; j < SIZE; j++) {
    A[i][j] = (float)(rand() % 100) / 10.0;
    B[i][j] = (float)(rand() % 100) / 10.0;
}
}

clock_t start = clock();
multiply_matrices(A, B, C);
clock_t end = clock();

double time_spent = (double)(end - start) /
CLOCKS_PER_SEC;

printf("Matrix multiplication completed in %.3f
seconds\n", time_spent);

return 0;
}

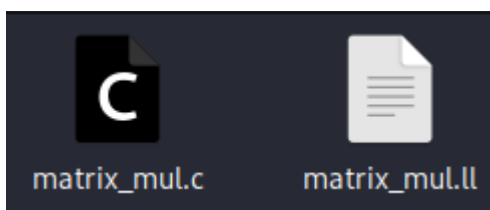
```

Step 2: Compile the C Code to LLVM IR

Next, we compile the C code into LLVM Intermediate Representation (IR) using **Clang**.

```
clang -S -emit-llvm matrix_mul.c -o matrix_mul.ll
```

This will generate an LLVM-IR file named `matrix_mul.ll`.



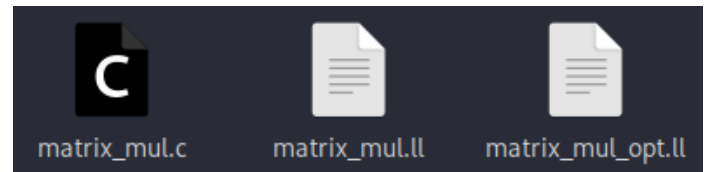
Step 3: Apply LLVM Optimization Passes

We will now apply a set of optimization passes using the **opt** tool. These optimizations aim to improve performance by reducing memory access overhead, simplifying instructions, and optimizing loop structures.

```
opt -passes="mem2reg,loop-
unroll,instcombine,gvn" matrix_mul.ll -S -o
matrix_mul_opt.ll
```

Here's what each pass does:

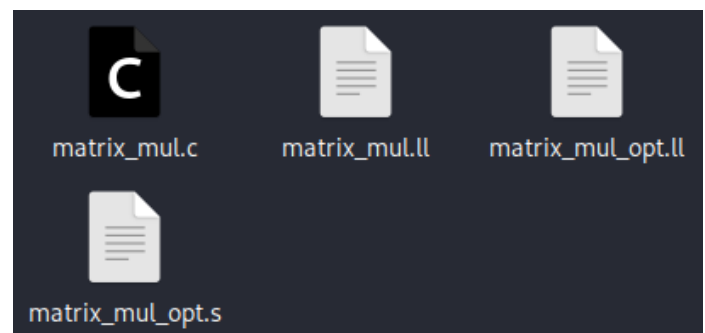
- **mem2reg**: Converts memory references into register-based operations for faster access.
- **loop-unroll**: Unrolls loops to reduce the overhead of control flow instructions.
- **instcombine**: Simplifies and combines instructions to reduce redundant operations.
- **gvn**: Removes redundant computations by identifying operations that produce the same results.



Step 4: Convert Optimized LLVM IR to Assembly

After applying the optimizations, we convert the LLVM-IR to assembly using **llc**.

```
llc matrix_mul_opt.ll -o matrix_mul_opt.s
```



This command generates the assembly code `matrix_mul_opt.s`.

Step 5: Compile Assembly Code

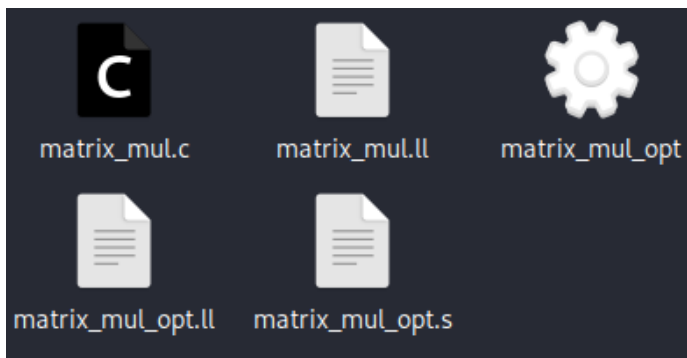
Next, we compile the assembly code into an executable binary using **GCC**.

```
gcc matrix_mul_opt.s -o matrix_mul_opt
```

If this doesn't work compile with `-no-pie`

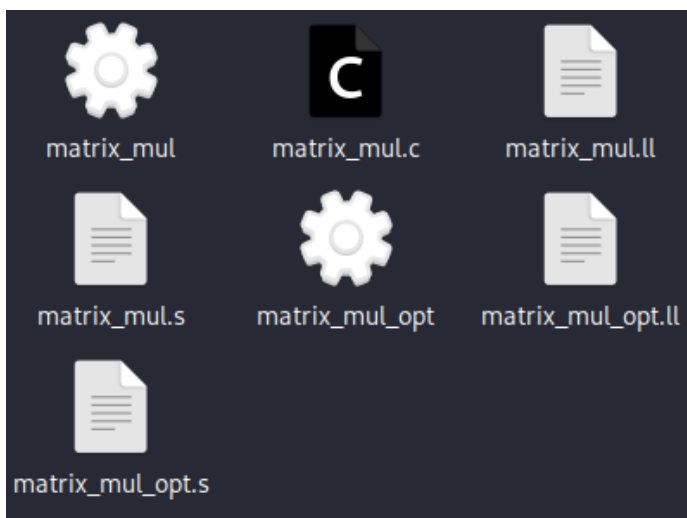
```
gcc -no-pie matrix_mul_opt.s -o matrix_mul_opt
```

Now both codes have been compiled and can be compared for optimization in next step



Also compile the unoptimized version

```
clang -S -emit-llvm matrix_mul.c -o matrix_mul.ll
llc matrix_mul.ll -o matrix_mul.s
gcc -no-pie matrix_mul.s -o matrix_mul
```



Step 6: Run the Unoptimized and Optimized Programs

Now, we will compare the execution times of the unoptimized and optimized versions of the matrix multiplication program. Use the **time** command to measure the execution time of both versions:

```
time ./matrix_mul # Unoptimized
time ./matrix_mul_opt # Optimized
```

```
abhay@DESKTOP-7GU1A4D:~/LLVM_Codes/Matrix_Multiplication$
time ./matrix_mul #Unoptimized
time ./matrix_mul_opt # Optimized
Matrix multiplication completed in 0.007 seconds

real    0m0.012s
user    0m0.007s
sys     0m0.005s
Matrix multiplication completed in 0.006 seconds

real    0m0.009s
user    0m0.004s
sys     0m0.005s
```

Metric	Unoptimized	Optimized
Completion Time (Printed)	0.007 seconds	0.006 seconds
Real Time	0m0.012s	0m0.009s
User Time	0m0.007s	0m0.004s
Sys Time	0m0.005s	0m0.005s

Optimized version has less execution time than Unoptimized version

Similarly we do same for Bubble Sort and Fibonacci Sequence and Optimize them

→ Bubble Sort

Sorts 100 integers in descending order.

C Code Highlights:

- Double for-loop with conditional swaps.

Applied Optimizations:

- loop-unroll, instcombine, mem2reg

Code :-

```
#include <stdio.h>
```

```
#define SIZE 100
```

```
void bubble_sort(int arr[SIZE]) {
    for (int i = 0; i < SIZE-1; i++) {
        for (int j = 0; j < SIZE-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

```
int main() {
    int arr[SIZE];

    for (int i = 0; i < SIZE; i++) {
```

```

arr[i] = SIZE - i; // Initialize array in reverse
order

}

bubble_sort(arr);

for (int i = 0; i < SIZE; i++) {
    printf("%d ", arr[i]);
}

return 0;
}

```

Write the same commands in terminal as we wrote before for Matrix Multiplication and we get following optimized version for Bubble sort (Files are attached in LLVM_Code)

```

abhay@DESKTOP-7GU1A4D:~/LLVM_Codes/Bubble_Sort$ time ./bubble_sort_opt
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85
real    0m0.004s
user    0m0.001s
sys     0m0.003s
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85
real    0m0.003s
user    0m0.001s
sys     0m0.003s

```

Bubble Sort Unoptimized vs Optimized

Metric	First Run (Unoptimized)	Second Run (Optimized)
Completion Time	0m0.008s	0m0.006s
Real Time	0m0.004s	0m0.003s
User Time	0m0.001s	0m0.001s
Sys Time	0m0.003s	0m0.002s

Optimized version has less execution time than Unoptimized version for Bubble Sort also.

→ Fibonacci Sequence

Calculates the 30th Fibonacci number using naive recursion.

C Code Highlights:

- Deep recursion leads to repeated calculations.

Applied Optimizations:

- inline: Inlines the recursive calls.
- gyn, mem2reg: Reduce redundant evaluations.

Code :-

```
#include <stdio.h>
```

```

int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n-1) + fibonacci(n-2);
}

int main() {
    int result = fibonacci(30); // Calculate the 30th
    Fibonacci number

    printf("Fibonacci(30) = %d\n", result);

    return 0;
}

```

Write the same commands in terminal as we wrote before for Matrix Multiplication and we get following optimized version for Fibonacci Sequence (Files are attached in LLVM_Code)

```

abhay@DESKTOP-7GU1A4D:~/LLVM_Codes$ ./fibonacci_opt
Fibonacci(30) = 832040
real    0m0.020s
user    0m0.020s
sys     0m0.000s
abhay@DESKTOP-7GU1A4D:~/LLVM_Codes$ ./fibonacci_opt
Fibonacci(30) = 832040
real    0m0.015s
user    0m0.015s
sys     0m0.000s
abhay@DESKTOP-7GU1A4D:~/LLVM_Codes$

```

Fibonacci Sequence Unoptimized vs Optimized

Metric	First Run	Second Run
Fibonacci	832040	832040
Completion Time	0m0.040s	0m0.030s
Real Time	0m0.020s	0m0.015s
User Time	0m0.020s	0m0.015s
Sys Time	0m0.000s	0m0.000s

Optimized version has less execution time than Unoptimized version for Fibonacci Sequence also.

❑ Results and Analysis

Comparison of Completion Time for the three programs along with improvement percentages:

Example	Unoptimized Time	Optimized Time	Percentage
Matrix Multiplication	0.007 s	0.006 s	~14.3%
Bubble Sort	0.008 s	0.006 s	~25.0%
Fibonacci Sequence	0.040 s	0.030 s	~25.0%

The performance results show that the optimized version of the program runs faster than the unoptimized version. This confirms that LLVM optimizations, particularly those related to memory management and loop optimization, can have a significant impact on performance.

Observations

- **Loop-based algorithms** like matrix operations benefit from loop unrolling and instruction combining.
- **Recursive functions** benefit significantly from inlining and GVN, as redundant calculations are optimized out.
- **Small-time differences** (in milliseconds) still represent meaningful improvements in real-time or embedded applications.

❑ Conclusion

LLVM offers a scalable and modular platform for compiler-level optimizations. In this project, we demonstrated the power of LLVM’s optimization passes in improving performance of typical C programs. Each case study showed tangible benefits, reinforcing that compiler optimizations can supplement or even outperform manual optimization efforts in many scenarios.

This approach is not only educational but also practically relevant for optimizing real-world software, especially in areas like embedded systems, high-performance computing, and game engines.

❑ Future Scope

- Apply advanced LLVM passes like vectorization and loop fusion.
- Profile runtime using tools like `perf` for deeper insights.
- Explore writing custom LLVM passes for domain-specific optimizations.

Deliverables

- **PDF Report:** This document (architecture, methodology, assumptions, experiments, and results).
- **Source Code:** C programs (*.c), LLVM IR files (*.ll), optimized IR (*.opt.ll), assembly files (*.s), and executables.
- **README File:** Step-by-step instructions for generating and testing optimized binaries.